

Universidad Nacional de Colombia  
Facultad de Ingeniería  
Ingeniería de Software

**Tutorial: Construcción de una aplicación backend desde cero**

Java + Spring Boot + Hibernate + PostgreSQL  
Proyecto OdontoGate

2026

## 1. Tecnologías y herramientas utilizadas

Para el desarrollo del presente tutorial se seleccionaron las siguientes tecnologías, cuya combinación representa una de las pilas tecnológicas más utilizadas en el desarrollo backend empresarial en la actualidad.

| Categoría                   | Tecnología                      | Versión        |
|-----------------------------|---------------------------------|----------------|
| Lenguaje de programación    | Java                            | 21             |
| Framework principal         | Spring Boot                     | 3.5.14         |
| ORM                         | Hibernate (via Spring Data JPA) | 6.6.49         |
| Base de datos               | PostgreSQL                      | 18 (Docker)    |
| Gestor de dependencias      | Maven                           | 3.9.15         |
| Entorno de desarrollo (IDE) | IntelliJ IDEA Community         | 2026.1.1       |
| Cliente de pruebas          | Postman                         | Última versión |

Nota: Spring Data JPA incluye Hibernate automáticamente como su implementación del estándar JPA. Al declarar esta dependencia en el archivo pom.xml, Maven descarga Hibernate sin necesidad de agregarlo por separado.

## 2. Librerías de ORM utilizadas

El mapeo objeto-relacional (ORM) es la técnica que permite representar las tablas de una base de datos como clases en el lenguaje de programación orientado a objetos. En este proyecto se utilizó el siguiente conjunto de librerías, todas incluidas automáticamente al agregar la dependencia spring-boot-starter-data-jpa al archivo pom.xml:

| Librería                     | Función                                                                |
|------------------------------|------------------------------------------------------------------------|
| spring-boot-starter-data-jpa | Dependencia principal que agrupa e integra JPA, Hibernate y HikariCP   |
| hibernate-core 6.6.49        | Implementación del ORM. Traduce las clases Java a tablas en PostgreSQL |

| Librería                      | Función                                                                      |
|-------------------------------|------------------------------------------------------------------------------|
| jakarta.persistence-api 3.1.0 | Estándar JPA que define las interfaces que Hibernate implementa              |
| HikariCP 6.3.3                | Pool de conexiones. Gestiona y reutiliza las conexiones a la base de datos   |
| postgresql 42.7.10            | Driver JDBC que permite la comunicación entre Java y PostgreSQL              |
| lombok 1.18.46                | Genera automáticamente getters, setters y constructores mediante anotaciones |

### 3. Configuración y levantamiento de la base de datos

La base de datos PostgreSQL se ejecuta dentro de un contenedor Docker, lo que permite un entorno de desarrollo reproducible e independiente del sistema operativo del desarrollador. Para crear e iniciar el contenedor se puede crear en la misma aplicación de Docker o se ejecuta el siguiente comando en la terminal:

```
docker run --name OdontoGate_DB \
  -e POSTGRES_USER=Administrador \
  -e POSTGRES_PASSWORD=12345 \
  -e POSTGRES_DB=OdontoGate_DB \
  -p 5432:5432 \
  -d postgres:latest
```

Cada parámetro del comando cumple una función específica en la configuración del contenedor, tal como se describe a continuación:

| Parámetro                      | Descripción                                                         |
|--------------------------------|---------------------------------------------------------------------|
| --name OdontoGate_DB           | Asigna un nombre identificable al contenedor                        |
| -e POSTGRES_USER=Administrador | Define el usuario de la base de datos                               |
| -e POSTGRES_PASSWORD           | Establece la contraseña del usuario                                 |
| -e POSTGRES_DB                 | Indica el nombre de la base de datos que se creará automáticamente  |
| -p 5432:5432                   | Mapea el puerto 5432 del contenedor al puerto 5432 del equipo local |

| Parámetro       | Descripción                                            |
|-----------------|--------------------------------------------------------|
| -d              | Ejecuta el contenedor en segundo plano (modo detached) |
| postgres:latest | Especifica la imagen de PostgreSQL a utilizar          |

#### 4. Contenido del archivo application.yml

El archivo application.yml es el archivo de configuración central de la aplicación. Spring Boot lo lee al iniciar y establece los parámetros de conexión a la base de datos, el comportamiento de Hibernate y el puerto del servidor. Este archivo se ubica en la ruta src/main/resources/application.yml.

Para su creación, el archivo application.properties generado por defecto por Spring Initializr fue renombrado a application.yml. Cabe señalar que Spring Initializr ofrece la opción de seleccionar el formato YAML directamente en el campo Configuration al momento de crear el proyecto, lo cual genera el archivo con la extensión correcta desde el inicio.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/OdontoGate_DB
    username: Administrador
    password: 12345
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true

server:
  port: 8080
```

A continuación se describe el propósito de cada propiedad configurada:

| Propiedad           | Descripción                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------|
| url                 | Dirección de conexión JDBC que indica el tipo de base de datos, el host, el puerto y el nombre de la BD |
| username / password | Credenciales del usuario de PostgreSQL definidas al crear el contenedor Docker                          |
| driver-class-name   | Clase del driver JDBC que Java utilizará para comunicarse con PostgreSQL                                |
| ddl-auto: update    | Indica a Hibernate que cree o actualice las tablas automáticamente según las entidades definidas        |
| show-sql: true      | Hace visible en la consola el SQL generado y ejecutado por Hibernate                                    |
| dialect             | Informa a Hibernate que el motor de base de datos es PostgreSQL                                         |
| format_sql: true    | Formatea el SQL mostrado en consola para facilitar su lectura                                           |
| port: 8080          | Puerto en el que el servidor Tomcat embebido recibirá las peticiones HTTP                               |

## 5. Creación del proyecto con Spring Initializr

Spring Initializr es la herramienta oficial para generar la estructura base de un proyecto Spring Boot. Se accede a través de la dirección:

<https://start.spring.io>

La configuración utilizada para el proyecto OdontoGate fue la siguiente:

| Campo       | Valor seleccionado |
|-------------|--------------------|
| Project     | Maven              |
| Language    | Java               |
| Spring Boot | 3.5.14             |
| Group       | com.odontogate     |
| Artifact    | Odontogate         |

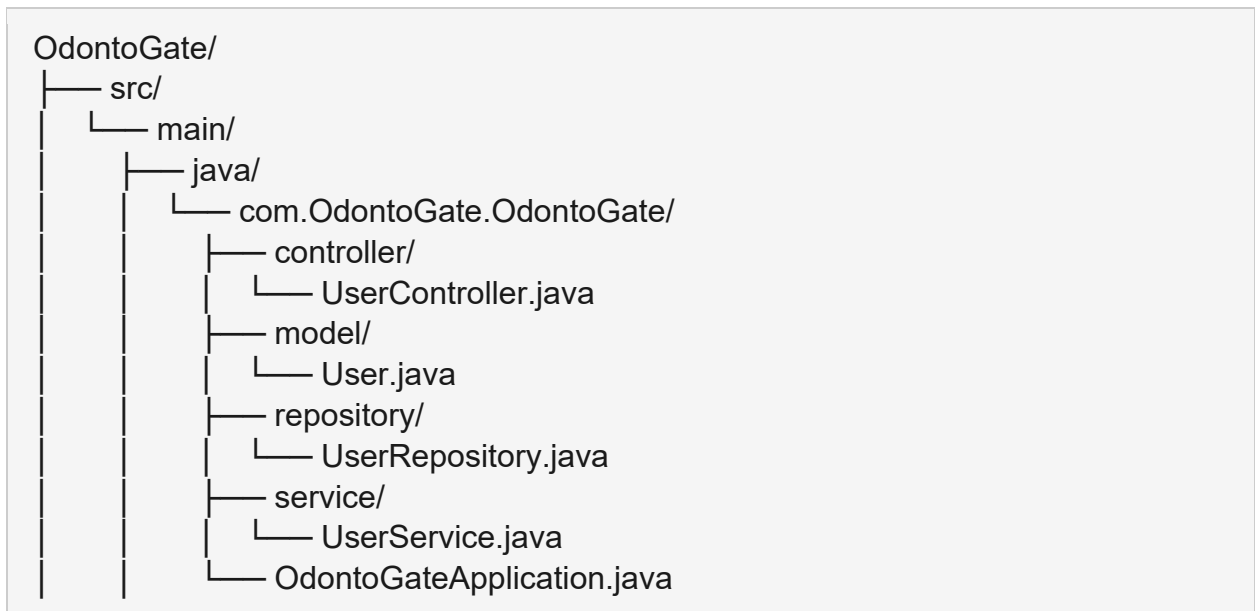
| Campo         | Valor seleccionado |
|---------------|--------------------|
| Packaging     | Jar                |
| Java          | 21                 |
| Configuration | YAML               |

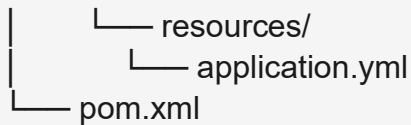
Las dependencias agregadas al momento de generar el proyecto fueron las siguientes:

- Spring Web: habilita la creación de endpoints REST mediante el servidor Tomcat embebido.
- Spring Data JPA: integra Hibernate y el estándar JPA para el mapeo objeto-relacional.
- PostgreSQL Driver: proporciona el conector JDBC necesario para la comunicación con PostgreSQL.
- Lombok: simplifica el código eliminando la necesidad de escribir manualmente getters, setters y constructores.

## 6. Estructura del proyecto

Al importar el proyecto en IntelliJ IDEA, la estructura de directorios generada es la siguiente:





Cada componente de esta estructura cumple un rol específico dentro de la arquitectura de la aplicación:

| Archivo / Carpeta              | Función                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------|
| OdontoGateApplication.java     | Punto de entrada de la aplicación. Contiene el método main() que inicia Spring Boot |
| model/User.java                | Entidad JPA que representa la tabla user en la base de datos                        |
| repository/UserRepository.java | Interfaz que gestiona las operaciones de acceso a datos mediante JpaRepository      |
| service/UserService.java       | Capa de lógica de negocio que media entre el controlador y el repositorio           |
| controller/UserController.java | Define los endpoints REST que reciben y responden peticiones HTTP                   |
| application.yml                | Archivo de configuración central de la aplicación                                   |
| pom.xml                        | Archivo de Maven que declara las dependencias y plugins del proyecto                |

## 7. Instanciación de entidades con Hibernate

En el contexto de JPA e Hibernate, una entidad es una clase Java que se mapea directamente a una tabla de la base de datos. Cada atributo de la clase corresponde a una columna de la tabla, y cada instancia del objeto representa un registro.

Para este tutorial se implementó la entidad User, que corresponde a la tabla user de la base de datos del proyecto OdontoGate. Esta tabla fue elegida por ser la base del modelo de datos, de la cual heredan las tablas patient, doctor y administrator.

### 7.1 Código de la entidad

```
package com.odontogate.odontogate.model;
```

```

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "\"user\"")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "name")
    private String name;

    @Column(name = "lastname")
    private String lastname;

    @Column(name = "email", unique = true)
    private String email;

    @Column(name = "password")
    private String password;

    @Column(name = "phone")
    private String phone;

    @Column(name = "active")
    private Boolean active;

    @Column(name = "created_at")
    private LocalDateTime createdAt;
}

```

## 7.2 Anotaciones utilizadas

| Anotación        | Descripción                                                                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| @Entity          | Indica a Hibernate que esta clase representa una tabla en la base de datos                                                                        |
| @Table(name=...) | Especifica el nombre exacto de la tabla en PostgreSQL. Las comillas adicionales son necesarias porque user es una palabra reservada en PostgreSQL |



| Anotación                 | Descripción                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------|
| @Data                     | Anotación de Lombok que genera automáticamente getters, setters, equals, hashCode y toString       |
| @Id                       | Designa el campo como llave primaria de la tabla                                                   |
| @GeneratedValue(IDENTITY) | Indica que el valor del ID será generado automáticamente por la base de datos de forma incremental |
| @Column                   | Mapea el atributo de la clase con una columna específica de la tabla                               |

Al ejecutar la aplicación con la propiedad `ddl-auto: update`, Hibernate lee la clase `User` y genera automáticamente la instrucción SQL para crear la tabla en PostgreSQL, sin necesidad de escribirla manualmente.

## 8. Arquitectura en capas

Spring Boot promueve el uso de una arquitectura en capas que separa las responsabilidades de cada componente. El flujo de una petición HTTP sigue la siguiente secuencia:

Cliente (Postman) → Controller → Service → Repository → Base de datos

### 8.1 UserRepository

La interfaz `UserRepository` extiende `JpaRepository`, lo que permite a Spring Boot generar automáticamente los métodos de acceso a datos más comunes sin necesidad de implementarlos manualmente.

```
package com.odontogate.odontogate.repository;

import com.odontogate.odontogate.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepository extends JpaRepository<User, Integer> {
}
```

Al extender `JpaRepository<User, Integer>`, Spring genera automáticamente métodos como `findAll()`, `save()`, `findById()` y `deleteById()`, equivalentes a las operaciones `SELECT`, `INSERT`, `UPDATE` y `DELETE` en SQL.

## 8.2 UserService

La capa de servicio contiene la lógica de negocio y actúa como intermediaria entre el controlador y el repositorio.

```
package com.odontogate.odontogate.service;

import com.odontogate.odontogate.model.User;
import com.odontogate.odontogate.repository.UserRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public User createUser(User user) {
        return userRepository.save(user);
    }
}
```

## 8.3 UserController

El controlador define los endpoints de la API REST y recibe las peticiones HTTP entrantes. La anotación `@RequestMapping` establece la URL base para todos los endpoints de este controlador.

```
package com.odontogate.odontogate.controller;

import com.odontogate.odontogate.model.User;
import com.odontogate.odontogate.service.UserService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {
```

```

private final UserService userService;

public UserController(UserService userService) {
    this.userService = userService;
}

@GetMapping
public ResponseEntity<List<User>> getAllUsers() {
    return ResponseEntity.ok(userService.getAllUsers());
}

@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
    return ResponseEntity.ok(userService.createUser(user));
}
}

```

## 9. Ejecución del proyecto

Con el contenedor Docker en ejecución y el proyecto correctamente configurado, la aplicación se inicia desde IntelliJ IDEA haciendo clic en el botón de ejecución ubicado en la esquina superior derecha, o directamente desde el archivo `OdontoGateApplication.java`. Al iniciarse correctamente, la consola de IntelliJ muestra el siguiente mensaje que confirma el funcionamiento de cada componente:

*Started OdontoGateApplication in 7.471 seconds (process running for 8.716)*

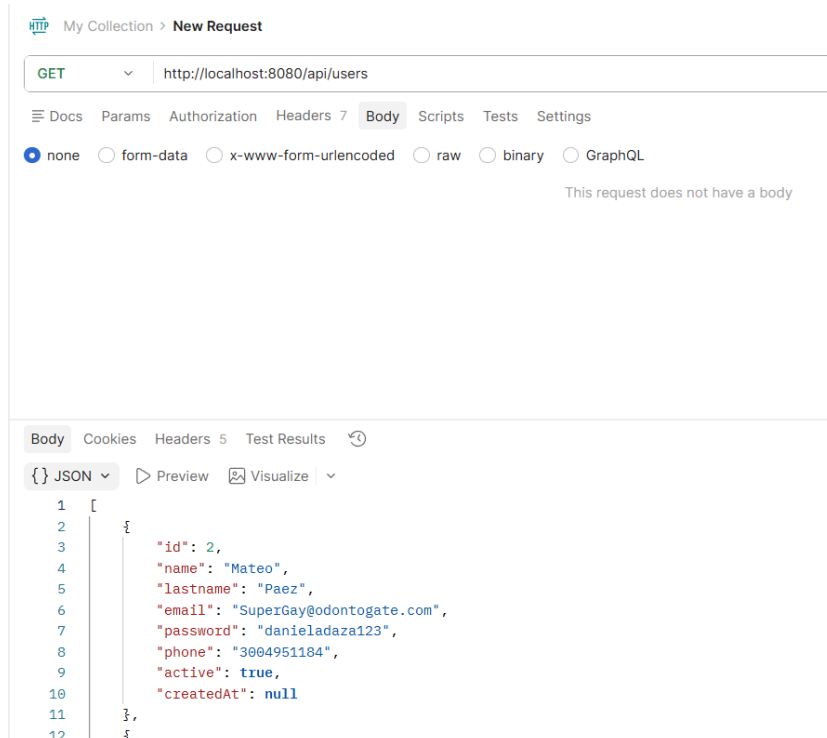
Este mensaje confirma que la conexión a la base de datos fue establecida exitosamente, que Hibernate generó automáticamente la tabla `user` en PostgreSQL y que el servidor está escuchando peticiones en el puerto 8080.

## 10. Prueba desde Postman

Postman es una herramienta que permite enviar peticiones HTTP a una API y verificar sus respuestas. Para este tutorial se realizaron dos pruebas que demuestran el funcionamiento completo del backend.

### 10.1 Consulta de usuarios (GET)

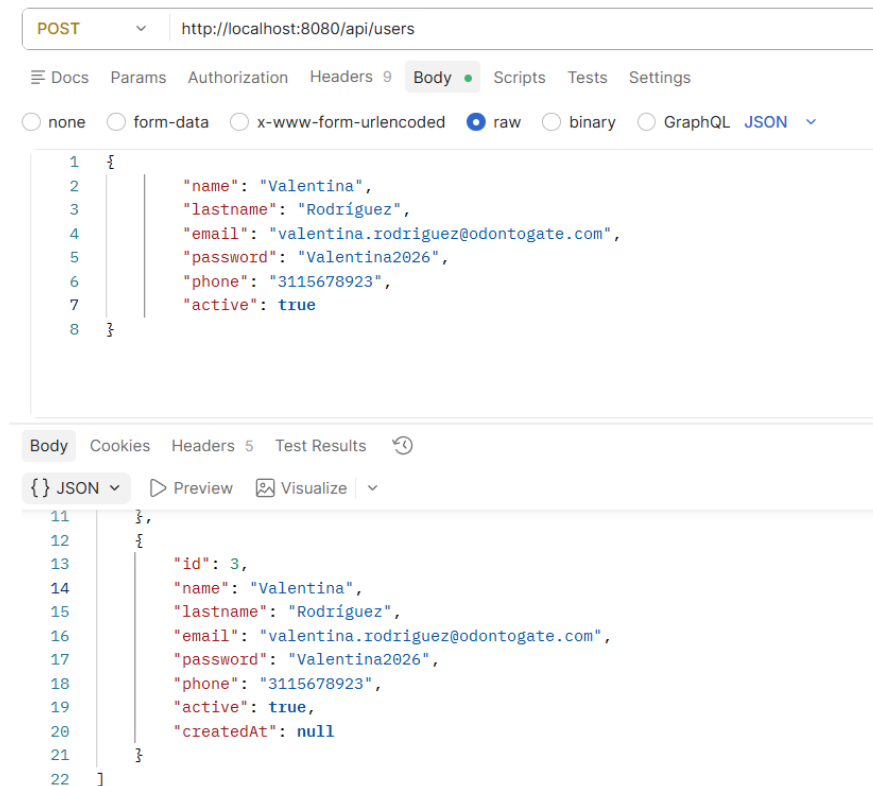
Esta petición permite obtener la lista de todos los usuarios registrados en la base de datos.



La respuesta inicial es un arreglo vacío, lo cual es el resultado esperado dado que la base de datos fue creada recientemente y no contiene registros. Si ya fueron agregados algunos usuarios, entonces aparecerán enlistados. El código de estado 200 OK confirma que la conexión y la consulta se ejecutaron correctamente.

## 10.2 Creación de un usuario (POST)

Esta petición permite registrar un nuevo usuario en la base de datos enviando los datos en formato JSON en el cuerpo de la solicitud.



El campo id con valor 1 fue generado automáticamente por PostgreSQL gracias a la anotación `@GeneratedValue` en la entidad User y así sucesivamente con el resto de usuarios. La respuesta con código 200 OK confirma que el registro fue guardado correctamente en la base de datos. Al realizar nuevamente la petición GET, el usuario recién creado aparecerá en el listado, demostrando la persistencia de los datos.

### 10.3 Eliminación de un usuario (DELETE)

Esta petición permite eliminar algún usuario que se encuentre en la base de datos, en su defecto, entonces mandará un error de que no se encontró al usuario.

DELETE ▼ http://localhost:8080/api/users/3

☰ Docs Params Authorization Headers 7 **Body** Scripts Tests Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

**Body** Cookies Headers 5 Test Results 🕒

📄 Raw ▼ ▶ Preview 🖼️ Visualize ▼

1 Usuario eliminado con éxito